

Lab Assignments 1

Task 1: Frequency Response using the DFT

```
import numpy as np
import matplotlib.pyplot as plt

def compute_dft_response(filter_coefficients):
    # Compute DFT of the filter coefficients
    H = np.fft.fft(filter_coefficients, n=2048) # Zero-pad to improve frequency resolution
    return H

# Define filter coefficients for H(z) = 1 + z^{-1}
fir_coefficients = [1, 1] # FIR filter coefficients

# Compute the DFT of the filter response
H_dft = compute_dft_response(fir_coefficients)

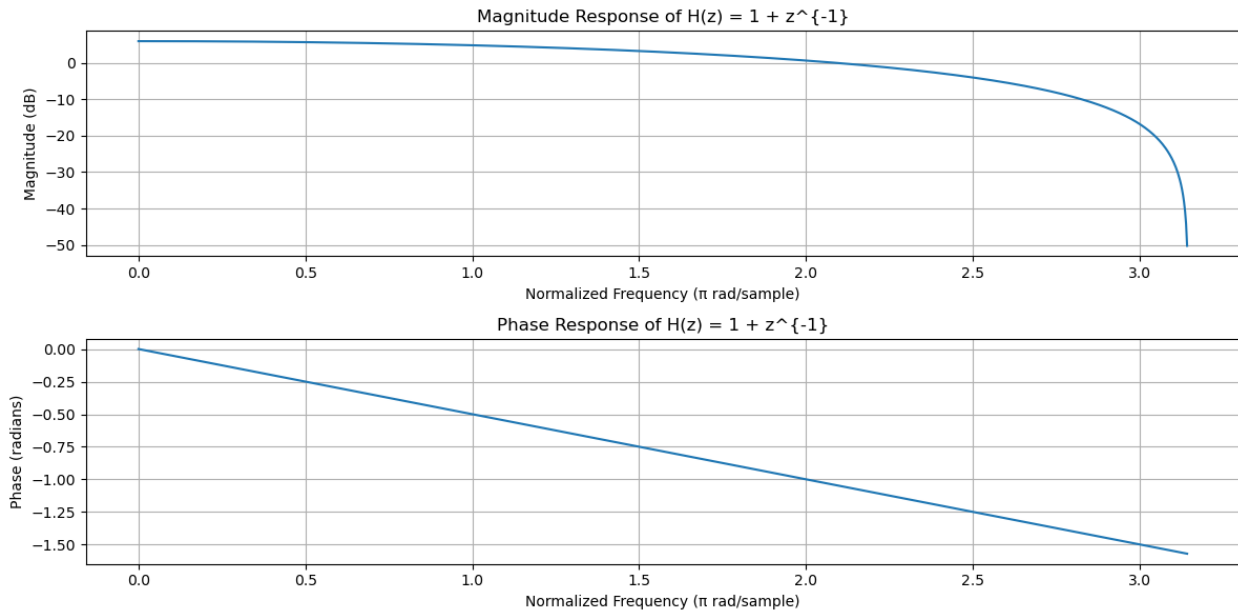
# Frequency axis
frequency = np.linspace(0, np.pi, len(H_dft) // 2)

# Plot the magnitude response
plt.figure(figsize=(12, 6))

# Magnitude response
plt.subplot(2, 1, 1)
plt.plot(frequency, 20 * np.log10(np.abs(H_dft[:len(H_dft) // 2])))
plt.title('Magnitude Response of H(z) = 1 + z^{-1}')
plt.xlabel('Normalized Frequency (π rad/sample)')
plt.ylabel('Magnitude (dB)')
plt.grid()

# Phase response
plt.subplot(2, 1, 2)
plt.plot(frequency, np.angle(H_dft[:len(H_dft) // 2]))
plt.title('Phase Response of H(z) = 1 + z^{-1}')
plt.xlabel('Normalized Frequency (π rad/sample)')
plt.ylabel('Phase (radians)')
plt.grid()

plt.tight_layout()
plt.show()
```



Task 2: FIR Filter Design using the Window Method

```
# Import necessary libraries
import numpy as np
import matplotlib.pyplot as plt

# Task 1: FIR Filter Design using Window Method
def fir_filter_window(N, omega_c):
    n = np.arange(0, N)
    h = omega_c / np.pi * np.sinc(omega_c / np.pi * (n - (N - 1) / 2))
    h = h * np.hamming(N) # Apply Hamming window
    return h

# Define parameters
N = 51
omega_c = 0.4 * np.pi

# Design filter
h_fir = fir_filter_window(N, omega_c)

# Compute the DFT of the filter response
H_dft = compute_dft_response(h_fir)

# Frequency axis
frequency = np.linspace(0, np.pi, len(H_dft) // 2)

# Plot the magnitude and phase responses
plt.figure(figsize=(12, 6))
```

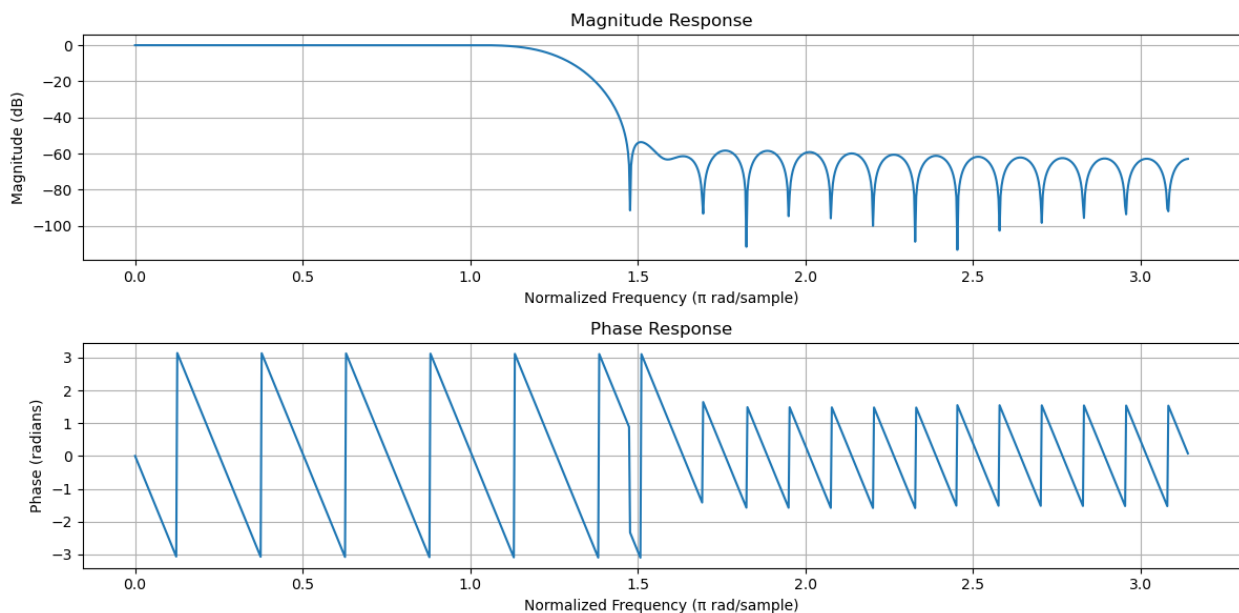
```

# Magnitude response
plt.subplot(2, 1, 1)
plt.plot(frequency, 20 * np.log10(np.abs(H_dft[:len(H_dft) // 2])))
plt.title('Magnitude Response')
plt.xlabel('Normalized Frequency ( $\pi$  rad/sample)')
plt.ylabel('Magnitude (dB)')
plt.grid()

# Phase response
plt.subplot(2, 1, 2)
plt.plot(frequency, np.angle(H_dft[:len(H_dft) // 2]))
plt.title('Phase Response')
plt.xlabel('Normalized Frequency ( $\pi$  rad/sample)')
plt.ylabel('Phase (radians)')
plt.grid()

plt.tight_layout()
plt.show()

```



Task 3: Linear-Phase Behavior of FIR Filters

```

def is_linear_phase(filter_coefficients):
    # Check if filter is symmetric (linear-phase)
    return np.all(filter_coefficients == filter_coefficients[::-1])

# Check linear-phase property of the designed filter
linear_phase_check = is_linear_phase(h_fir)
print(f'Is the FIR filter linear-phase? {linear_phase_check}')

Is the FIR filter linear-phase? True

```

Task 4: FIR Filter Application on a Signal using Convolution

```
def apply_fir_filter(signal, filter_coefficients):  
    return np.convolve(signal, filter_coefficients, mode='same')  
  
# Create a test signal  
n = np.arange(0, 100)  
signal = np.sin(0.2 * np.pi * n) + 0.5 * np.sin(0.7 * np.pi * n)  
  
# Apply FIR low-pass filter  
filtered_signal = apply_fir_filter(signal, h_fir)  
  
# Plot original and filtered signals  
plt.figure(figsize=(12, 6))  
plt.plot(n, signal, label='Original Signal')  
plt.plot(n, filtered_signal, label='Filtered Signal', linestyle='--')  
plt.title('Original and Filtered Signals')  
plt.xlabel('Sample Index')  
plt.ylabel('Amplitude')  
plt.legend()  
plt.grid()  
plt.show()
```

